data might *also* contain line breaks. Simply wrapping `str_getcsv()` (to parse the fields) around `explode()` (to break multi-line data into an array of strings) can (and usually does) lead to broken data.

sscanf()

Now we come to quite a powerful function, `sscanf()`. It belongs to a family of functions (such as `sprintf()` et al., which we'll see more of in this section) with a rich history in computer programming. The BCPL language (first implemented in 1967) contained a function `WRITEF()` which took a specially formatted string expression and interpolated further argument data into a returned string, and the C language has had a `scanf()` function in `<stdio.h>` since the days of Version 7 Unix. PHP's `sscanf()` wraps the C equivalent and will extract portions of an input string according to a formatting expression. The function also accepts any number of additional variables, which will have the extracted data assigned to them (they are passed by reference by default); otherwise, an array of the extracted data is returned. This sounds complicated (and it can certainly get complicated), so let's start with a simple example:

```
$str = 'Gender: M, Height: 1.85m, Weight: 95kg';
var_dump(sscanf($str, 'Gender: %1s, Height: %fm, Weight: %dkg'));
```

```
Output:
array(3) {
  [0]=>
  string(1) "M"
  [1]=>
  float(1.85)
  [2]=>
  int(95)
}
```

The second argument to `scscanf()` is the formatting expression, and the parts of the expression triggering the extraction are `%1s`, `%f`, and `%d`. These are *conversion specifications*, which have the syntax `%` followed by certain meta-characters to describe width, precision, or to control other behavior, and then a *specifier* ("d" for decimal integer, "f" for float, "s" for string, and so on). In our expression above, `%1s` matches "a single string byte," `%f` matches "number with a decimal point," and `%d` matches an